Estudo Dirigido — Tratamento de Exceções em Rust

Conceitos de Linguagens de Programação

Aluno: Gustavo Marcos Brendler

Outubro de 2026

1 Implementação

O programa recebe o nome do arquivo pela linha de comando (`cargo run - dados/correto.txt`), lê cada linha no formato `nome;nota1;nota2`, calcula a média e imprime o resultado. Os erros foram divididos em dois níveis:

- **Erros de E/S** (abrir ou ler o arquivo): representados por `std::io::Error` e propagados com `?` até `main`, que decide a mensagem e encerra o programa (não há como continuar sem o arquivo).
- **Erros de registro** (campo ausente, nota inválida, campos a mais): representados por um tipo próprio, o `enum ErroRegistro`, e tratados linha a linha com `match`, de modo que um registro inválido é apenas ignorado e os seguintes continuam sendo processados.

```rust
use std::env;
use std::fmt;
use std::fs::File;
use std::io::{self, BufRead, BufReader};
use std::num::ParseFloatError;
use std::process;

/// Erros possíveis no processamento de UM registro (uma linha).
/// Cada variante corresponde a uma "classe de exceção" de uma
/// linguagem com tratamento tradicional.
#[derive(Debug)]
enum ErroRegistro {
    /// Um dos campos esperados não está presente na linha.
    CampoAusente { campo: &'static str },
    /// A linha possui mais campos que o esperado.
    CamposExcedentes,
    /// Um campo de nota não pôde ser convertido para f64.
    NotaInvalida {
        campo: &'static str,
        valor: String,
        causa: ParseFloatError,
    },
}

impl fmt::Display for ErroRegistro {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        match self {
            ErroRegistro::CampoAusente { campo } => {
                write!(f, "campo ausente: '{}'", campo)
            }
            ErroRegistro::CamposExcedentes => {
```

```
                write!(f, "registro possui mais de três campos")
            }
            ErroRegistro::NotaInvalida { campo, valor, causa } => {
                write!(f, "valor inválido em '{}': \"{}\" ({})", campo, valor, causa)
            }
        }
    }
}

struct Estudante {
    nome: String,
    nota1: f64,
    nota2: f64,
}

impl Estudante {
    fn media(&self) -> f64 {
        (self.nota1 + self.nota2) / 2.0
    }
}

/// Obtém o próximo campo da linha.
/// `next()` devolve Option<&str>: None indica que o campo não existe.
/// O `match` converte esse Option em Result, associando um erro ao None
/// (campo vazio também é tratado como ausente).
fn proximo_campo<'a, I>(campos: &mut I, nome: &'static str) -> Result<&'a str,
    ErroRegistro>
where
    I: Iterator<Item = &'a str>,
{
    match campos.next() {
        Some(texto) if !texto.trim().is_empty() => Ok(texto.trim()),
        _ => Err(ErroRegistro::CampoAusente { campo: nome }),
    }
}

/// Converte o texto de uma nota para f64.
/// `parse` devolve Result<f64, ParseFloatError>; `map_err` traduz o erro
/// de baixo nível para o erro do domínio da aplicação.
fn converter_nota(texto: &str, campo: &'static str) -> Result<f64, ErroRegistro> {
    texto.parse::<f64>().map_err(|causa| ErroRegistro::NotaInvalida {
        campo,
        valor: texto.to_string(),
        causa,
    })
}

/// Analisa uma linha e constrói um Estudante.
/// Cada `?` propaga imediatamente o Err para quem chamou esta função.
fn analisar_linha(linha: &str) -> Result<Estudante, ErroRegistro> {
    let mut campos = linha.split(';');

    let nome = proximo_campo(&mut campos, "nome")?;
    let nota1 = converter_nota(proximo_campo(&mut campos, "nota1")?, "nota1")?;
    let nota2 = converter_nota(proximo_campo(&mut campos, "nota2")?, "nota2")?;

    if campos.next().is_some() {
        return Err(ErroRegistro::CamposExcedentes);
    }
```

```rust

    Ok(Estudante {
        nome: nome.to_string(),
        nota1,
        nota2,
    })
}

/// Processa o arquivo inteiro. Erros de E/S (abrir/ler o arquivo) são
/// propagados com `?`; erros de registro são tratados aqui mesmo com
/// `match`, permitindo continuar nas linhas seguintes.
/// Retorna (registros válidos, registros inválidos).
fn processar_arquivo(caminho: &str) -> io::Result<(usize, usize)> {
    let arquivo = File::open(caminho)?;
    let leitor = BufReader::new(arquivo);

    let mut validos = 0;
    let mut invalidos = 0;

    for (indice, linha) in leitor.lines().enumerate() {
        let linha = linha?;
        let numero = indice + 1;

        if linha.trim().is_empty() {
            continue;
        }

        match analisar_linha(&linha) {
            Ok(estudante) => {
                println!("{:<10} média: {:5.2}", estudante.nome, estudante.media());
                validos += 1;
            }
            Err(erro) => {
                eprintln!("[linha {}] registro ignorado -- {}  => \"{}\"", numero,
    erro, linha);
                invalidos += 1;
            }
        }
    }

    Ok((validos, invalidos))
}

fn main() {
    let caminho = match env::args().nth(1) {
        Some(c) => c,
        None => {
            eprintln!("Uso: estudo-excecoes <arquivo>");
            process::exit(2)
        }
    };

    println!("Processando '{}'...", caminho);

    match processar_arquivo(&caminho) {
        Ok((validos, invalidos)) => {
            println!(
                "Concluído: {} registro(s) válido(s), {} inválido(s).",
                validos, invalidos
```

```
            );
        }
        Err(erro) if erro.kind() == io::ErrorKind::NotFound => {
            eprintln!("Erro: o arquivo '{}' não foi encontrado.", caminho);
            process::exit(1);
        }
        Err(erro) => {
            eprintln!("Erro ao ler '{}': {}", caminho, erro);
            process::exit(1);
        }
    }
}

#[cfg(test)]
mod testes {
    use super::*;

    #[test]
    fn registro_valido() {
        let e = analisar_linha("Maria;8.5;7.0").unwrap();
        assert_eq!(e.nome, "Maria");
        assert_eq!(e.media(), 7.75);
    }

    #[test]
    fn nota_invalida() {
        assert!(matches!(
            analisar_linha("Pedro;dez;8.5"),
            Err(ErroRegistro::NotaInvalida { campo: "nota1", .. })
        ));
    }

    #[test]
    fn campo_ausente() {
        assert!(matches!(
            analisar_linha("Pedro;10.0"),
            Err(ErroRegistro::CampoAusente { campo: "nota2" })
        ));
    }

    #[test]
    fn campos_excedentes() {
        assert!(matches!(
            analisar_linha("Pedro;10.0;8.0;9.0"),
            Err(ErroRegistro::CamposExcedentes)
        ));
    }

    #[test]
    fn arquivo_inexistente() {
        let erro = processar_arquivo("inexistente.txt").unwrap_err();
        assert_eq!(erro.kind(), io::ErrorKind::NotFound);
    }
}
```

2 Execução dos casos de teste

```
$ cargo run -q -- dados/correto.txt
Processando 'dados/correto.txt'...
Maria      média:  7.75
Joao       média:  7.50
Pedro      média:  9.25
Ana        média:  6.75
Carlos     média:  8.50
Concluído: 5 registro(s) válido(s), 0 inválido(s).
(código de saída: 0)

$ cargo run -q -- dados/inexistente.txt
Processando 'dados/inexistente.txt'...
Erro: o arquivo 'dados/inexistente.txt' não foi encontrado.
(código de saída: 1)

$ cargo run -q -- dados/numero_invalido.txt
Processando 'dados/numero_invalido.txt'...
Maria      média:  7.75
Joao       média:  7.50
[linha 3] registro ignorado -- valor inválido em 'nota1': "dez" (invalid float literal)  => "Pedro
    ;dez;8.5"
Ana        média:  6.75
Carlos     média:  8.50
Concluído: 4 registro(s) válido(s), 1 inválido(s).
(código de saída: 0)

$ cargo run -q -- dados/campo_ausente.txt
Processando 'dados/campo_ausente.txt'...
Maria      média:  7.75
Joao       média:  7.50
[linha 3] registro ignorado -- campo ausente: 'nota2'  => "Pedro;10.0"
Ana        média:  6.75
Carlos     média:  8.50
Concluído: 4 registro(s) válido(s), 1 inválido(s).
(código de saída: 0)
```

Mensagens de erro são escritas na saída de erro (`eprintln!`) e o resumo final na saída padrão. Além dos quatro casos, o programa inclui testes unitários (`cargo test`), todos aprovados.

3 Questões

1. Como cada problema foi representado e tratado?

Problema	Representação	Tratamento
Arquivo inexistente	`File::open` devolve `Err(io::Error)` com `kind() == ErrorKind::NotFound`.	Propagado com `?` em `processar_arquivo`; tratado em `main` por um `match` com guarda que testa o `kind()`; exibe mensagem e encerra com código 1.
Valor numérico inválido	`"dez".parse::<f64>()` devolve `Err(ParseFloatError)`, convertido por `map_err` em `ErroRegistro::NotaInvalida`.	Propagado com `?` de `converter_nota` até `analisar_linha` e desta até o laço; ali um `match` registra o erro e o laço segue para a próxima linha.
Campo ausente	`split(';').next()` devolve `None` (um `Option`), que é convertido em `Err(ErroRegistro::CampoAusente)`.	Mesmo caminho do caso anterior: propagado por `?` e tratado no `match` do laço, sem interromper o processamento.

Em Java essas situações corresponderiam, respectivamente, a `FileNotFoundException`, `NumberFormatException` e `ArrayIndexOutOfBoundsException` (ou uma exceção própria). Em Rust nenhuma delas é uma exceção: todas são *valores* comuns de retorno. O *tipo* da função (`Result<Estudante, ErroRegistro>`, `io::Result<...>`) anuncia quais erros podem ocorrer, e o compilador não permite usar o valor obtido sem antes considerar o caso de erro. Vale notar que a ausência de campo nasce como `Option` ("não há valor", sem motivo associado) e só vira erro quando o programa decide que essa ausência é um problema e lhe dá um significado (`CampoAusente { campo: "nota2"}`).

2. Uma operação que retorna `Result<T, E>`

A função `converter_nota` tem tipo `fn(&str, &'static str) -> Result<f64, ErroRegistro>`:

```
fn converter_nota(texto: &str, campo: &'static str) -> Result<f64, ErroRegistro> {
    texto.parse::<f64>().map_err(|causa| ErroRegistro::NotaInvalida {
        campo, valor: texto.to_string(), causa,
    })
}
```

- `Ok(f64)`: a conversão foi bem-sucedida; o valor carregado é a nota numérica (ex.: `"8.5"` → `Ok(8.5)`).
- `Err(ErroRegistro)`: a conversão falhou; o valor carregado descreve a falha — qual campo, qual texto e a causa original (ex.: `"dez"` → `Err(NotaInvalida { campo: "nota1", valor: "dez", ... })`).

Relação com o levantamento de exceção. Na videoaula, levantar uma exceção (`throw`/`raise`) significa sinalizar que uma condição anormal ocorreu, interrompendo o fluxo normal e transferindo o controle para um tratador, junto com um objeto que descreve a exceção. Em Rust, *retornar `Err(...)`* cumpre o papel de sinalização: a função termina e entrega ao chamador um objeto descritivo do erro (o análogo do objeto de exceção, com "tipo" e dados). A diferença é que nada é desviado automaticamente: o `Err` é um valor de retorno normal, que segue pelo caminho usual de retorno de função. O "levantamento" é, portanto, explícito e local, e o tipo `Result` torna a possibilidade de erro parte da assinatura — algo parecido com as exceções *verificadas* de Java (`throws`), porém imposto pelo sistema de tipos e não por uma regra à parte.

3. O operador ?

O ? é usado em `analisar_linha` e em `processar_arquivo`:

```
let nota1 = converter_nota(proximo_campo(&mut campos, "nota1")?, "nota1")?;
let arquivo = File::open(caminho)?;
```

- **Quando a expressão produz `Ok(v)`**: o ? "desembrulha" o valor e a expressão inteira vale `v`. A execução continua normalmente na linha seguinte (`nota1` recebe o `f64`; `arquivo` recebe o `File`).
- **Quando produz `Err(e)`**: a função atual retorna *imediatamente* `Err(From::from(e))` ao seu chamador; o restante do corpo não é executado. O `From::from` permite converter automaticamente o tipo de erro, se houver conversão definida.

Ou seja, `x?` equivale a:

```
match x { Ok(v) => v, Err(e) => return Err(From::from(e)) }
```

Comparação com propagação de exceções. No modelo tradicional, quando uma unidade não possui tratador para a exceção levantada, ela é encerrada e a exceção é *propagada* automaticamente para o chamador (seguindo a cadeia dinâmica de chamadas), até encontrar um tratador ou chegar ao topo e abortar o programa. O ? produz o mesmo efeito observável — a função é encerrada e o erro "sobe" para o chamador —, porém com diferenças importantes:

- a propagação é **explícita**: cada ponto em que um erro pode sair da função está marcado com ?; em Java, qualquer chamada pode silenciosamente propagar uma exceção não verificada;
- ela ocorre **um nível por vez**: cada função intermediária precisa declarar um tipo de retorno compatível (`Result`) e usar ? de novo; não há "salto" por várias pilhas de uma só vez;
- não existe desempilhamento especial: é um `return` comum.

No programa, a nota inválida "dez" é propagada de `converter_nota` → `analisar_linha` → laço em `processar_arquivo`, onde é tratada; já o `NotFound` vai de `File::open` → `processar_arquivo` → `main`.

4. Tratamento explícito com `match` versus `try-catch`

Rust

```
match analisar_linha(&linha) {
  Ok(estudante) => {
    println!("{} média: {:.2}", ...);
  }
  Err(erro) => {
    eprintln!("[linha {}] ... {}",
              numero, erro);
    invalidos += 1;
  }
}
```

Java (equivalente)

```
try {
  Estudante e = analisarLinha(linha);
  System.out.printf(...);
} catch (NumberFormatException ex) {
  System.err.println(...);
  invalidos++;
} catch (CampoAusenteException ex) {
  System.err.println(...);
  invalidos++;
}
```

Onde está o tratador e como é associado ao erro.

- **Java:** o tratador é a cláusula `catch` associada a um bloco `try`. A associação é *dinâmica* e feita pelo *tipo* da exceção: quando algo é lançado em qualquer ponto do bloco protegido (inclusive dentro de métodos chamados por ele), a execução é desviada e a JVM procura, entre os `catch` desse `try` e depois dos `try` envolventes na pilha de chamadas, o primeiro cujo tipo é compatível. O código protegido e o código de tratamento ficam separados.
- **Rust:** o "tratador" é o braço `Err(erro) => ...` do `match`, e ele se associa a *um valor específico* — o resultado daquela única chamada `analisar_linha(&linha)`. A associação é

estática e léxica: é o próprio programador que inspeciona o valor retornado, no ponto exato da chamada. A seleção entre tipos de erro se faz por casamento de padrões (ex.: `Err(e) if e.kind() == NotFound` em `main`, ou `Err(ErroRegistro::CampoAusente {..})`), e o compilador exige que o `match` seja exaustivo, ou seja, que `Ok` e `Err` sejam ambos considerados.

Em ambos os casos, após o tratamento a execução continua na instrução seguinte ao `try/match` — o modelo de *terminação*, não de retomada: o registro com erro não é reprocessado, o laço apenas segue para a próxima linha.

5. Rust possui tratamento de exceções no sentido da disciplina?

Não, para erros recuperáveis. No sentido apresentado na disciplina — um mecanismo da linguagem no qual uma exceção é *levantada*, desvia implicitamente o fluxo de controle e é *propagada automaticamente* pela cadeia de chamadas até um tratador associado dinamicamente — Rust não oferece esse mecanismo para erros esperados. Esses erros são tratados como **valores** (`Result` e `Option`), com apoio de `match`, do operador `?` e de combinadores (`map_err`, `ok_or`, `unwrap_or`, ...).

O que mais se aproxima de exceção em Rust é o `panic!`: ele interrompe o fluxo e desempilha a pilha (*unwinding*) executando destrutores, e pode até ser capturado com `std::panic::catch_unwind`. Porém, ele se destina a erros *irrecuperáveis* (bugs, violação de invariantes), pode ser configurado para simplesmente abortar (`panic = "abort"`) e não é a forma idiomática de sinalizar erros como os deste estudo.

Aspecto	**Modelo tradicional (Java, C++, Python)**	**Rust**
Sinalização	`throw` de um objeto exceção	`return Err(valor)` (ou `None`)
Representação do erro	Objeto de uma hierarquia de classes de exceção	Valor de qualquer tipo, geralmente um `enum`
Fluxo de controle	Desvio implícito, não local	Retorno comum, local e explícito
Propagação	Automática pela pilha até achar tratador	Explícita com `?`, um nível por vez
Associação ao tratador	Dinâmica, pelo tipo, ao `catch` mais próximo	Léxica, por `match` sobre o valor retornado
Erro não tratado	Termina o programa (ou a thread)	Não passa despercebido: `Result` ignorado gera aviso do compilador, e para obter o `T` é preciso decidir o que fazer com o `Err` (`match`, `?`, `unwrap`...)
Visibilidade na assinatura	Só exceções verificadas (Java)	Sempre, no tipo de retorno
Continuação	Terminação	Terminação (a função que falha não é retomada)
Erros irrecuperáveis	Mesma construção (`Error`, `RuntimeException`)	Mecanismo separado: `panic!`

Semelhanças: ambos separam o resultado normal do resultado de erro; ambos carregam informação descritiva sobre a falha (tipo e dados); ambos permitem tratar o erro no local ou repassá-lo ao chamador; e ambos seguem o modelo de terminação, em que a unidade que detectou o erro não é retomada.

Diferenças: em Rust, o erro é um valor comum e não um evento de controle; a propagação é explícita (`?`) e não automática; a associação com o tratador é léxica e verificada pelo compilador (exaustividade do `match`), o que impede que um erro seja esquecido; e não há custo nem complexidade de desempilhamento para o caminho de erro. Em troca, o código fica um pouco mais verboso e cada função intermediária precisa declarar o tipo de erro que pode devolver — custo que o operador `?` e a conversão via `From` reduzem bastante.

Em resumo, Rust *resolve o mesmo problema* que o tratamento de exceções resolve (detectar, sinalizar, propagar e tratar situações anormais), mas por meio do sistema de tipos, e não por um mecanismo de desvio de controle. Seu único mecanismo semelhante a exceções, o `panic!`, é reservado para falhas irrecuperáveis.